Testing Reports — File Compression Tool (Huffman Coding, Python)

Item	Detail
Document	Testing Reports — Unit, System Validation, Code Coverage, Branch Coverage
Product	File Compression Tool — Huffman coding, Python (CLI + GUI)
Version	1.0
Date	6 November 2026
Repository	`KeerthanaKumaresh1/file-compression-tool`
Team (GitHub usernames)	KeerthanaKumaresh1 (Keerthana Kumaresh — repository owner), rishi2908, khushi309
Companion document	`Test_Plan.md` (IEEE-format Test Plan v1.0, 6 Nov 2026) — test cases UT_01–UT_10 are defined there and their results are summarised here

> **Full names:** Only Keerthana Kumaresh is named in full in the evidence provided. The other two team members are identified by GitHub username only (rishi2908, khushi309). Their full names were not in the evidence inspected — add them from the team list before submission if required.
>
> **What these reports do and do not claim:** Every number below is a measured number from the runs cited. No test result, PR number, or coverage number has been invented. Where a number was not in the evidence, it is stated as not in the evidence instead of guessed.

Contents

1. Unit Test Report
2. System Validation Report
3. Code Coverage Report (Line Coverage)
4. Branch Coverage Report
5. Gaps & Follow-up Actions

---

1. Unit Test Report

1.1 Summary

Item	Result
Test suite location	`tests/`

Item	Result
Test files	**15**
Tests total	**81**
Passed	**81**
Failed	**0**
Skipped	0 — none recorded in the evidence
Run time	**0.28 s** (first run) / **0.55–0.62 s** (coverage runs)
Python	**3.12.3**
Framework	`pytest`, `unittest` style (`unittest.TestCase` classes)
Verdict	**PASS — 81/81**

1.2 Per-file results

Table 1.1 — Tests per file. The counts sum to 81 (2+8+7+5+5+9+5+5+7+4+12+4+5+3 = 81). Every file passed all of its tests; there were 0 failures in every file.

Test file	Tests	Passed	Failed
`test_batch_compression.py`	2	2	0
`test_compression_stats.py`	8	8	0
`test_corruption.py`	7	7	0
`test_error_handler.py`	5	5	0
`test_file_format.py`	5	5	0
`test_history.py`	9	9	0
`test_huffman_code_table.py`	5	5	0
`test_huffman_codes.py`	5	5	0
`test_huffman_tree.py`	7	7	0
`test_integration.py`	4	4	0
`test_output_protection.py`	12	12	0
`test_performance.py`	4	4	0
`test_tree_visualizer.py`	5	5	0
`test_verification.py`	3	3	0
Total (15 files)	**81**	**81**	**0**

Notes on the table:

- Largest test file: `test_output_protection.py` (12 tests) — overwrite protection.
- Smallest: `test_batch_compression.py` (2 tests).

- `test_corruption.py` (7 tests) tests the **decompressor**, not `corruption.py` — see §5, gap G1. Its 7 passes do not mean `corruption.py` is covered; §3 shows `corruption.py` at 0% line coverage.
- There is no separate `test_frequency.py` file in the 15 files; frequency behaviour is exercised through the tree / codes / table tests and the live runs in §2.

1.3 How to reproduce

From the repository root, in the virtual environment (Python 3.12.3):

```
python -m pytest tests -q
```

Expected output: 81 passed, 0 failed, in about 0.28 s (without coverage) — the exact time will vary by machine; the pass count is the check, not the time.

1.4 Environment

Python 3.12.3, `pytest`, Linux test machine used for this run. No Windows or macOS run is recorded in the evidence, so no Windows/macOS result is claimed. Exact machine: Ubuntu 24.04.5 LTS, x86_64, 4 vCPU, 7.7 GiB RAM, Python 3.12.3.

1.5 What the unit tests cover — and what they do not

Covered: frequency analysis, Min-Heap, Huffman tree and codes, code table, tree visualisation, file format (magic `HFC1`, header `'>4sQIBH'`, CRC32, padding), file validation, decompression, lossless verification, compression statistics, performance, error handling (exit codes 2, 3, 4, 5, 6), overwrite protection (`'xb'`), batch compression, history (JSON, max 100 entries), and end-to-end integration (`test_integration.py`, 4 tests).

Not covered by unit tests: the GUI (`gui.py` — not imported by any test), and `corruption.py` (0% — see §3 and §5 G1).

---

2. System Validation Report

System validation is the live, end-to-end check of the real CLI on real files. These are **not** unit tests and are **not** counted in the 81. All three runs were performed on **6 November 2026** on the Linux test machine (Python 3.12.3).

2.1 Run (a) — Small file: `sample.txt`

Item	Detail
Input	`sample.txt` — content `AAAABBBCCD`, **10 bytes**
Command	`python main.py compress sample.txt --table --tree --verify --time` (from the repository root)
Output	`sample.txt.hfc` — **42 bytes**

Item	Detail
Compression ratio	**0.24:1**
Space saved	**−320%** (the file grows — see interpretation)
Average bits / symbol	**1.9**
Huffman codes	A=0, B=10, C=111, D=110
Huffman tree root	**10**
Frequencies	A=4, B=3, C=2, D=1
Verification	**Passed** (`verify` — see Run (c))
Time	**0.007 s**
Peak memory	**14.58 KB**
Verdict	**Pass**

Interpretation: the compressed file is larger than the input because the `.hfc` header, frequency table, padding and CRC32 overhead (32 bytes of overhead on a 10-byte input) exceeds the saving from Huffman coding. This is expected for such a small file and is not a failure. The 19-bit payload (4×1 + 3×2 + 2×3 + 1×3 = 19 bits; 19 ÷ 10 = 1.9 bits/symbol) is correct for these codes.

2.2 Run (b) — Large file: `big.txt`

Item	Detail
Input	`big.txt` — repetitive text, **90,000 bytes**
Command	`python main.py compress big.txt --verify --time` (from the repository root)
Output	50,914 bytes (`.hfc`)
Space saved	**43.43%**
Compression ratio	**1.77:1**
Average bits / symbol	**4.51**
Verification	**Passed**
Time	**0.107 s**
Throughput	**0.80 MB/s**
Peak memory	**205.57 KB**
Decompression check	Decompressed file **byte-identical to the original, confirmed with `cmp`**
Verdict	**Pass**

Interpretation: on repetitive text the tool saves 43.43% (90,000 → 50,914 bytes). The byte-identical `cmp` result is the strongest check in this report: decompression is lossless on a 90 KB file, not only on the 10-byte sample.

2.3 Run (c) — `verify` subcommand on the sample

Item	Detail
Input	`sample.txt.hfc` (from Run (a))
Command	`python main.py verify sample.txt sample.txt.hfc` (from the repository root)
Output	**"OK: The compressed file restores the original exactly."**
Verdict	**Pass**

2.4 System validation summary

Run	Input	Output	Lossless?	Verdict
(a) `sample.txt`	10 bytes	42 bytes, ratio 0.24:1, −320%	Yes — verify passed	Pass
(b) `big.txt`	90,000 bytes	50,914 bytes, ratio 1.77:1, 43.43% saved	Yes — `cmp` byte-identical	Pass
(c) `verify` on sample	`sample.txt.hfc`	"OK: The compressed file restores the original exactly."	Yes	Pass

Overall system-validation verdict: PASS (3/3 runs). The two business-level claims the tool makes — it compresses, and it restores the original exactly — are both confirmed on real files, small and large.

What system validation does not claim: only the two text inputs above were validated live. No binary, already-compressed, or >90 KB file was validated live, and no GUI run is recorded.

---

3. Code Coverage Report (Line Coverage)

3.1 Method

- Tool: `coverage.py`.
- Scope: **`--source=compressor`** — the `compressor` package only.
- Branch measurement: **off** for this section (branch coverage is §4).
- Run time for the coverage runs: 0.55–0.62 s.

3.2 Result — total

Measure	Value
Statements (total)	**641**

Measure	Value
Missed statements	**106**
Line coverage (total)	**83%**

3.3 Result — per module

Table 3.1 — Line coverage per module, `compressor` package only. Statement counts per module were not in the evidence provided (except `corruption.py`, 19 statements) and are not stated here; only the coverage percentages and the missed-line ranges that were in the evidence are given.

Module	Line coverage
`__init__.py`	100%
`batch_compression.py`	96%
`cli.py`	62% (missed lines 51–82, 86–100, 104–109, 116–118, 126–127)
`compression_stats.py`	92%
`corruption.py`	**0%** (19 statements, missed lines 2–32)
`decompressor.py`	85%
`error_handler.py`	82%
`errors.py`	100%
`file_format.py`	86%
`file_validation.py`	65%
`frequency.py`	100%
`history.py`	91%
`huffman_code_table.py`	82%
`huffman_codes.py`	100%
`huffman_tree.py`	100%
`min_heap.py`	90%
`output_protection.py`	89%
`performance.py`	100%
`pipeline.py`	97%
`tree_visualizer.py`	87%
`utils.py`	88%
`verification.py`	90%
TOTAL (`compressor` only)	**83% (641 statements, 106 missed)**

3.4 What is — and is not — included

- **Included:** the 22 modules of the `compressor` package listed in Table 3.1.
- **NOT included — `gui.py`:** the GUI file (`gui.py`, 785 lines) is **not** included in this coverage figure and is **not** imported by the tests. The 83% figure says nothing about the GUI.
- **NOT included — `tests/`:** the test files themselves are not counted in the `compressor`-only figure.

3.5 Warning — do NOT quote the 91% figure as product coverage

An earlier all-files coverage figure of **91%** was measured with `tests/` included. That figure counts the test code itself as "covered" and therefore inflates the result. **It must not be quoted as product coverage.** The product coverage figure is the one in §3.2–§3.3: **83% for the `compressor` package only.**

3.6 Interpretation

- Strong (≥90%): `__init__`, `batch_compression`, `compression_stats`, `errors`, `frequency`, `history`, `huffman_codes`, `huffman_tree`, `min_heap`, `performance`, `pipeline`, `verification`. The core Huffman algorithm (`frequency`, `huffman_tree`, `huffman_codes`) is at 100%.
- Middle (80–89%): `decompressor`, `error_handler`, `file_format`, `huffman_code_table`, `output_protection`, `tree_visualizer`, `utils`.
- Weak (<70%): `cli` (62%), `file_validation` (65%), and `corruption` (0% — see §5 G1). The `cli` misses are concentrated in the ranges listed in Table 3.1 — mostly CLI error / edge paths that the unit tests do not drive through the CLI entry point.

---

4. Branch Coverage Report

4.1 Method

- Tool: `coverage.py` with `--branch` and `--source=compressor`.
- Same test suite (81 tests) as §1.

4.2 Result — total

Measure	Value
Statements (total)	**641**
Missed statements	**106**
Branches (total)	**226**
Partial branches	**35**
Branch coverage (total)	**79%**

A "partial branch" is a branch (e.g. an `if` / loop condition) where only one of the two outcomes was exercised by the tests — for example, the true side was taken but the false side never was. 35 of 226 branches are partial.

4.3 Result — per module

Table 4.1 — Branch coverage per module, `compressor` package only.

Module	Branch coverage
`batch_compression.py`	94%
`cli.py`	50%
`compression_stats.py`	90%
`corruption.py`	0%
`decompressor.py`	80%
`error_handler.py`	82%
`errors.py`	100%
`file_format.py`	86%
`file_validation.py`	60%
`frequency.py`	100%
`history.py`	89%
`huffman_code_table.py`	79%
`huffman_codes.py`	91%
`huffman_tree.py`	100%
`min_heap.py`	88%
`output_protection.py`	92%
`performance.py`	97%
`pipeline.py`	92%
`tree_visualizer.py`	82%
`utils.py`	79%
`verification.py`	92%
`__init__.py`	100% line; 0 branches, so branch coverage is not applicable (100% by definition — nothing to miss).
TOTAL (`compressor` only)	**79% (641 stmts, 106 miss, 226 branches, 35 partial)**

4.4 Interpretation

- Branch coverage (79%) is lower than line coverage (83%), as expected: a line can be executed without both sides of its branch being taken.
- Strong (≥90%): `batch_compression`, `compression_stats`, `errors`, `frequency`, `huffman_codes`, `huffman_tree`, `output_protection`, `performance`, `pipeline`, `verification`. The core algorithm is again at or near 100%.
- Middle (79–89%): `decompressor`, `error_handler`, `file_format`, `history`, `huffman_code_table`, `min_heap`, `tree_visualizer`, `utils`.
- Weakest: **`cli` 50%**, **`file_validation` 60%**, **`corruption` 0%**. These three are the same weak modules as in line coverage, and for the same reasons: CLI branches are mostly driven only by the happy path in tests, `file_validation` edge branches are not all exercised, and `corruption.py` is never called at all (see §5 G1).
- The 35 partial branches are the concrete next target: each one is a condition whose untested side should get a test (see §5, action A2), rather than a reason to add more tests blindly.

---

5. Gaps & Follow-up Actions

Numbered, honest, in priority order. None of these changes the Pass verdicts in §§1–2, but each is a likely viva question.

#	Gap	Evidence	Follow-up action
G1	**`corruption.py` is 0% covered — dead code.** Its functions `validate_structure`, `check_payload_length`, `check_checksum` are never called by the pipeline / decompressor or by the tests.	Line 0% (19 stmts, miss 2–32); branch 0%. `test_corruption.py` (7 tests) tests the **decompressor**, not `corruption.py`.	**A1:** Either wire `corruption.py` into the pipeline / decompressor and add unit tests for its three functions, or remove the module and document that corruption detection lives in `decompressor.py` (magic, padding, duplicate/zero frequency, `sum(freq)` check, CRC32). Do not leave it as unexplained dead security code.
G2	**CLI coverage is low: 62% line, 50% branch** (miss 51–82, 86–100, 104–109, 116–118, 126–127).	Tables 3.1 and 4.1.	**A2:** Add CLI tests for every subcommand (`compress`, `decompress`, `verify`) and every exit code (2 validation, 3 creation, 4 corrupted, 5 output-exists, 6 verification), including the error paths. Re-measure; the 35 partial branches in §4 are the checklist for this work.
G3	**`file_validation` is 65% line / 60% branch.**	Tables 3.1 and 4.1.	**A3:** Add tests for the validation edge cases (empty file, unreadable file, unsupported type — as defined in the SRS). Record the

#	Gap	Evidence	Follow-up action
			empty-file CLI run (see Test Plan §5.1 SR-2).
G4	**GUI is untested and excluded from coverage** (`gui.py`, 785 lines, not imported by tests).	§3.4; Test Plan §2.2, §4.1.	**A4:** Perform and record at least one manual GUI run (compress → decompress → verify) before the demo, or state clearly in the demo that the GUI is manual-only. No manual GUI run was performed for these reports.
G5	**Whole-file in-memory processing; no large-file test.** Largest live input was 90,000 bytes.	§2; Test Plan §4.3, §10 R5.	**A5:** State the limit honestly in the demo. Do not claim >4 GB or low-memory support. (Optional: test one larger file, e.g. 10 MB, and record time / peak memory, if time allows before the viva.)
G6	**Small-file overhead looks like a failure if unexplained** (10 → 42 bytes, −320%).	§2.1; Test Plan UT_07/UT_10.	**A6:** Keep the overhead explanation (header + frequency table + CRC) in the demo script. Present the `big.txt` figures (43.43% saved) as the compression evidence, and the sample as the correctness evidence.
G7	**Earlier 91% all-files coverage figure could be misquoted.**	§3.5.	**A7:** Quote only the `compressor`-only figures: **83% line, 79% branch**. Do not quote 91% as product coverage.
G8	**Items not in the evidence** — PR numbers per assignee, video links, and a manual GUI run.	Test Plan §10 R7; §5 of this report.	**A8:** Check PR numbers and videos in the repository before submission; perform and record a manual GUI run, or state in the demo that the GUI is manual-only. Do not fill them from memory.

Overall verdict: Unit tests **81/81 Pass**; system validation **3/3 Pass** (both files lossless); line coverage **83%** and branch coverage **79%** for the `compressor` package, with the core Huffman algorithm at 100%. The product's core claim is proven; the gaps above are known, bounded, and actionable.

End of Testing Reports v1.0 — 6 November 2026.